

[[TOC]]

Capability description

This capability allows Creating entities from a Data model. Each row in the data model will become an entity.

#Prerequisites - Entity / relation definition must exist. - Properties without FieldVisibility should at least have the value |6|

#Entity Creation: ##Step 1: The entity Data model Creating a data model with the following columns: - ID - should be marked as 'isID' = 1 , MUST be unique - column for each entity property - Sys_LastUpdatedOn - Field Role: Sequence - Must have a display title defined

##Step 2: Indexing Service - Entity Creation 1. Add a new record in table 'indexing_sources' `SELECT * FROM sqlite_metadata.indexing_sources` - ID = Your Entity DmId (Id of the data model created in step1) - DATA_SOURCE_TYPE = "DataModel" - CONNECTION_ID = -12 (connection to orient) - INDEXING_INTERVAL= NULL - DESCRIPTION = just normal description - IS_ACTIVE = 1 (0 = wont be indexed) - BULK_COUNT = 100 (amount of records sent to entity creation - keep it 100)

2. Add a new record in table 'indexing_sources_stores' `SELECT * FROM sqlite_metadata.indexing_sources_stores`
- SOURCE_ID = Your Entity DmId (Id of the data model created in step1)
 - MORE_INFO = The entity ID
 - STORE_TYPE = 4 (Fixed value - 4 is the value for entity creation)
 - STORE_CONNECTION_ID = -100 (Fixed)
 - STORE_CONNECTION_IS_ENDPOINT = 0 (Fixed)
 - IDENTIFIERS_MAPPINGS_LIST = NULL
 - STORE_BULK_COUNT = 100

Important Note on STORE_BULK_COUNT configuration:

If STORE_BULK_COUNT in the indexing_sources_stores table is equal to or higher than the Data Model's page size, the indexing service will process only the first page of records and stop, even if more data exists. This may result in missing entities or relations.

Recommended configuration: - For entities, STORE_BULK_COUNT should always be set to 0 — unless the Data Model is continuously updated. - For relations, STORE_BULK_COUNT should be lower than the page size defined in the Data Model.

Example: | Page Size | BULK_COUNT | STORE_BULK_COUNT | Indexing
Result Behavior | | | | |

| | 100 | 100 | 100 | Processes only the first chunk (one iteration) | | 100 | 10 | 10
 | Processes a single small chunk | | 100 | 10 | 0 | Continues indexing until all
 data is processed |

Warning: If STORE_BULK_COUNT is not configured correctly, the indexing process may not complete as expected.

- IS_ACTIVE = 1
- TTIGGER_TYPE = What type of trigger you want :
 - 2 = Indexing Interval (The interval between the end of indexing to the next indexing)
 - 3 = Cron (scheduling the running time)
- INDEXING_INTERVAL = Time in seconds. For example, the value: 600 = 10 minuts
- INDEXING_CRON = What time the process will be run, For example, Every day at 09:15 = 0 15 09 ? * * for more options check here: <https://www.quartz-scheduler.org/documentation/quartz-2.3.0/tutorials/crontrigger.html>
- BATCH_ID - Setup up multiplate entity creation as a batch process in specific order. **works only with cron trigger type**
- BATCH_ORDER = Order of the indexing processes in a specific batch (the first process =1, second=2 ...)
- ADDITIONAL_PARAMS = mapping JSON for the entity properties
Dictionary:
 - Mode = 1 Entity, 2 Relation
 - Type = the entity type
 - DefinitionPropertyName - entity property name (EPxx_xx)
 - FilePropertyName - DM column
 - Scope = 1 normal entity property, 4 Key entity property (identification) ### Mapping JSON Sample Sample mapping should be set for the “Source Store” in the “*additionalinfo*” column:

```
{
  "id": null,
  "mode": 1,
  "type": "EN291",
  "filePath": null,
  "extension": null,
  "moreInfo": null,
  "mapping": [
    {
      "definitionPropertyName": "EP291_1",
```

```

        "filePropertyName": "ID",
        "scope": 4
    },
    {
        "definitionPropertyName": "EP291_2",
        "filePropertyName": "NAME",
        "scope": 1
    }
],
"orderNum": 0,
"newFileID": 0,
"originalFileName": null,
"tags": null,
"caseID": 0
}

```

##Step 3: Restart and Verify Restart the Indexing Service to start the creation process and Clear Server Cache through Admin Tools in the Web Client.

You may have a look at the table indexing_audit in order to verify. `SELECT * FROM sqlite_metadata.indexing_audit where index_source_ID=xxx` See item count and Items fetch values to see if any data was found and indexed:

Note: If the existing entity was found it will be updated, otherwise it will be created. If more than 1 entity is found existing, an update will occur for the first entity found.

#Relation Creation: This method allows the creation of links between different entity types, based on the values appearing in the entity property. you have to use values that can Identify the entity like name, address, number, and so on. > **Important Note!** > - In this Method - please make sure that you mark all fields in the entity properties as “Is Quarriable” and the entities have a display title > - In case the entity value appears more than once, and the properties are not defined as ID, a relation will be created on the first entity found.

##Step 1: The Relation Data model Creating the Data model with the following columns - example: - ID - Row ID = IsID, Must - Must have a display title field - LastUpdateDate = Sequence, must - From Entity = The column name for the source entity - Can use any column name - To Entity = The column name for the destination entity - Can use any column name - Relation property Name = Property in the relation, not mandatory.

Note: Don't Use “From” and “To” field names in the data model since they are reserved names.

##Step 2: Indexing Service - Relation Creation 1. Adding a new record in table 'indexing_sources' `SELECT * FROM sqlite_metadata.indexing_sources` - ID= Your Relation DmId (Id of the data model created in step1) -

DATA_SOURCE_TYPE = "DataModel" - CONNECTION_ID = -12
 (connection to orient) - INDEXING_INTERVAL= NULL - DESCRIPTION=
 just normal description
 - IS_ACTIVE=1 (0 = wont be indexed) - BULK_COUNT=1000 (amount of
 records sent to entity creation - keep it 100)

2. Add a new record in table 'indexing_sources_stores' **SELECT * FROM
 sqlite_metadata.indexing_sources_stores**

- SOURCE_ID = Your Relation DmId (Id of the data model created in
 step1)
- MORE_INFO = The Relation ID
- STORE_TYPE = 5 (Fixed value - 5 is the value for relation creation)
- STORE_CONNECTION_ID = -100 (Fixed)
- STORE_CONNECTION_IS_ENDPOINT = 0 (Fixed)
- IDENTIFIERS_MAPPINGS_LIST = NULL
- STORE_BULK_COUNT = 100
- IS_ACTIVE = 1
- TTIGGER_TYPE = What type of trigger you want :
 - 2 = Indexing Interval (The interval between the end of indexing to
 the next indexing)
 - 3 = Cron (scheduling the running time)
- INDEXING_INTERVAL = Time in seconds. For example, the value: 600
 = 10 minuets
- INDEXING_CRON = What time the process will be run, For
 example, Every day at 09:15 = 0 15 09 ? * * for more options
 check here:<https://www.quartz-scheduler.org/documentation/quartz-2.3.0/tutorials/crontrigger.html>
- BATCH_ID - Setup up multiplate entity creation as a batch process in
 specific order. **works only with cron trigger type**
- BATCH_ORDER = Order of the indexing processes in a specific batch
 (the first process =1, second=2 ...)
- ADDITIONAL_PARAMS = mapping JSON for the entity properties
Dictionary:
 - Mode = 1 Entity, 2 Relation
 - Type = the Relation ID (for example: RL16)
 - DefinitionPropertyName - entity property name (EPxx_xx)
 - FilePropertyName -The entity property like EP16_1 or RP100_2

– Scope = 1-Relation property, 2-From entity, 3-To entity The JSON:

```
{
  "mode": 2,
  "type": "RLxx",
  "caseID": 0,
  "mapping": [
    {
      "scope": 2,
      "filePropertyName": "Name1",
      "definitionPropertyName": "EPxx_x"
    },
    {
      "scope": 3,
      "filePropertyName": "Name2",
      "definitionPropertyName": "EPxx_x"
    },
    {
      "scope": 1,
      "filePropertyName": "test",
      "definitionPropertyName": "RPxx_x"
    }
  ],
  "orderNum": 0
}
```

Step 3: Restart and Verify Restart the Indexing Service to start the creation process and Clear Server Cache through Admin Tools in the Web Client.

Related Tables:

indexing_audit_status_mv open the table : `SELECT * FROM sqlite_metadata.indexing_audit_status_mv`; This table shows the last index time for each data model. If you need to change the last index time, you need to do it from this table. During the indexing service work, the service uses the 'last index time' from this table to determine if new data needs to be indexed (changed behavior from using the 'last index time' from the indexing_audit table)

indexing_tasks_audit open the table : `SELECT * FROM sqlite_metadata.indexing_tasks_audit`; This table shows indications of errors in the indexing processes per indexing source. Meaning, if there is an error in the indexing process of creating entities or relations, the error will appear there.

Troubleshoot:

ITEM_FETCHED = 0

- Make sure your DM is quarriable.
- Make sure your DM has IsID, Main Display and Sequence fields.

ITEM_COUNT = 0

- Invalid JSON mapping/parameters name in JSON - check for the error in the indexing service
- Invalid values in the JSON mapping - check for the error in the entities service.

Invalid Chars

Note that the chars below are not supported in the ID field - please make sure your data doesn't contain them.

] / /n (**break line**)

In additional, IDs with the chars below will not be updated, a new entity created insted.

' ['